What is a terminal ?

A **terminal** (or *shell*) is a text interface to talk directly to your operating system — no mouse, no windows, just commands.

Depending on your OS:

- Linux / Mac: **Bash** or **zsh** (already installed)
- Windows: **PowerShell**, or better, **WSL** (Ubuntu)

It may look intimidating at first, but you only need a handful of commands to get started — and it will quickly feel natural.

Exercise 0

Open a terminal on your machine. You should see a **prompt**, *e.g.* `alice@machine:~$`. Type `pwd` and hit Enter. What do you see ?

What is a script ?

A **script** is simply a plain text file containing a sequence of instructions that `Python` reads and executes **from top to bottom**.

Concretely, it is a file with a `.py` extension — nothing more. You can write it in any text editor, and run it from the terminal with `python my_script.py`.

A script is typically written to **automate a repetitive task**, for instance:

- Renaming 300 files according to a naming convention
- Sorting a messy `Downloads` folder by file type
- Sending a daily summary email from a CSV file
- Scraping a webpage and saving the results to a file

In this class, we will see some basics regarding the terminal and how to build scripts that can access files & folders outside of `Python`. In the next (and last) PS, you will build a script to tidy a messy folder.

Some terminal basic commands

Command	Action	Example
`ls / dir`	List files in current folder	`ls /Downloads`
`cd` *`folder`*	Move into a folder	`cd Documents`
`cd ..`	Go up one level	`cd ..`
`pwd`	Print current location	`pwd`
`mkdir` *`name`*	Create a new folder	`mkdir results`
`mv` *`src dst`*	Move or rename a file	`mv a.txt b.txt`
`cp` *`src dst`*	Copy a file	`cp a.txt backup/`
`rm` *`file`*	Delete a file (no undo!)	`rm old.log`
`touch` *`name`*	Create a file	`touch test.py`

Use `dir` instead of `ls`, and `copy` / `move` instead of `cp` / `mv` in PowerShell (classic). WSL users can use the Unix commands directly.

Exercise 1

Create a folder `OOPCourse` inside `Documents` with two subfolders `PracticalSessions` and `Slides`.

Options & arguments

Anatomy of a command: `command [options] argument(s)`

An **argument** is the *target* of the command (a file, a URL etc.). It's like the x in $f(x)$. For example `mkdir name` creates a folder with name $x =$ `name`.

An **option** (or *flag*) modifies its behaviour, usually starting with `-` or `--`. Usually a single dash, so called *short flag*, is a shorthand notation for *long flag* (*i.e.* with double dash). Probably the most useful option is `-h` or `--help`.

Exercise 2

Try `wget -h` (and also `wget -help`). It's possible that `wget` does not exist depending on your OS. It should work on  and  . On iOS, try `curl`.

Example: downloading a file from the web.

The command `curl` (or `wget`) downloads a file from the web:

```
# Basic usage: curl <url>
$ curl https://example.com/file.pdf
# -O renames the downloaded file
$ curl -O my_name.pdf https://example.com/file.
    pdf
# -q runs silently (no output)
$ curl -s -O my_name.pdf https://example.com/
    file.pdf
```

For instance, the flag `-O` can also be written using the long flag `--remote-name`, or `-s` as `--silence`.

Exercise 3

Create a `Slides` subfolder inside `OOPCourse` and download the slides of this course at `https://rodriguel.github.io/data/docs/SlidesOOP.pdf` there. Rename it as `slides.pdf`.

Running `Python` in interactive mode in a terminal

If you type `python` in a terminal, `Python` REPL shows up.

> *A **read–eval–print loop** (REPL), also termed an **interactive toplevel** or **language shell**, is a simple interactive computer programming environment that takes single user inputs, executes them, and returns the result to the user; a program written in a REPL environment is executed piecewise.*

Exercise 4

Open Python's REPL and compute `2 + 2`. To exit, type `exit()`.

❗ REPL is nice to do quick experiments with `Python`, but we cannot *save* its input / output, *reuse* or either *share* your code that way. That's why you are better using a **script** in these contexts, namely to save your code in a file `myProgram.py` (*i.e.* the script) and let `Python` run it.

Running a Python script

Given a script `my_script.py`, you can execute it in a terminal with `python my_script.py`. For example:

```
# Content of my_script.py
s = "Hello world !"
print(s)
```

then if you go with your terminal to the folder **where this file is located** this file and type

```
python my_script.py
```

you should see `Hello world !` returned & printed to your terminal session. *Good job, you've built your first script !* ☺

Exercise 5

Create a script that asks for the user to enter its name (*e.g.* `Roger`) and that greets him / here (*i.e.* prints `Hello Roger !`). Execute it in a terminal session then.

The pathlib library

Let us introduce the pathlib library in Python. It's part of the Python standard library since Python 3.4, which is by now quite old, so it should be available on your machine.

It is used to represents **file paths as objects** (*That's OOP !*)

```python
from pathlib import Path

# Create a Path object (i.e. absolute path)
p = Path("/home/alice/Documents")

# Relative path
p = Path(".")  # current directory (where Python is lauched)
p = Path("my_folder/file.txt")
```

Some useful functionalities

A `Path` object contains some useful **attributes** (*That's OOP !*):

```python
from pathlib import Path

p = Path("/home/alice/Documents/report.pdf")

print(p.name)          # report.pdf
print(p.stem)          # report
print(p.suffix)        # .pdf
print(p.parent)        # /home/alice/Documents
```

You can build new paths with the / operator:

```python
# Build a path with /
new = p.parent / "archives" / "report_v2.pdf"
print(new)
# /home/alice/Documents/archives/report_v2.pdf
```

Some useful functionalities *yet*

For a `Path` object `p`, you can check *it exists*, if *it is a file* or a *folder*:

```
p.exists() # returns True / False if whether or
    not the file/folder exists.
p.is_file() # idem ... a file or not
p.is_dir() # idem .... a folder or not
```

To **list the contents** of a folder `p = Path("path/to/folder")`, we can list is content (i.e. like the `ls` command line) with `p.iterdir()`

```
for item in folder.iterdir():
  print(item.name)
```

Exercise 6

List the contents of a folder of your choice on your own machine (use REPL).

Some useful functionalities *yet again*

To create a folder with path `p` (which is a `Path` object) you can use the `mkdir` **object method**:

```
# Create a folder (parents=True: creates
    intermediates)
# exist_ok=True: no error if it already exists
new = Path("results/2024/january")
new.mkdir(parents=True, exist_ok=True)
```

You can also *read / write* a text file `p = Path("notes.txt")`:

```
content = p.read_text(encoding="utf-8")
print(content)
# Write (overwrites existing content)
p.write_text("New line\n", encoding="utf-8")
# Append to the end
with p.open("a", encoding="utf-8") as f:
    f.write("One more line\n")
```

The shutil library

While pathlib handles **paths** and simple operations, shutil library handles **copying and moving** file content.

```
import shutil
from pathlib import Path

src = Path("report.pdf")
dst = Path("archives/report.pdf")
```

To **copy** src to dst:

```
shutil.copy(src, dst)
```

To **cut** src to dst:

```
shutil.move(src, dst)
```

To **delete** an entire folder & its contents :

```
shutil.rmtree(Path("temp_folder"))
```

Some exercises !

Exercise 7

Create this folder structure using `pathlib` only: `project/data/raw` and `project/data/processed`. Check that they exist with `.exists()` afterwards.

Exercise 8

Write a script that creates a file `log.txt`, writes three lines into it (the name of three files of your choice), then reads it back and prints each line prefixed with a line number, *e.g.* `1. photo.jpg`.

Exercise 9

Using `shutil` and `pathlib`: create a folder `inbox/` containing three empty files (`a.txt`, `b.txt`, `c.txt` — use `.touch()`). Then copy `a.txt` into a new folder `backup/`, move `b.txt` into `backup/` as well, and finally delete `inbox/` entirely.

The `sys.argv` library to pass arguments to a script

A useful script should adapt **without being edited**. The `sys.argv` retrieves arguments passed on the command line.

Consider the following script, saved in `my_script.py`.

```python
import sys
# sys.argv[0] = script name
# sys.argv[1] = first argument, etc.
print("Script:", sys.argv[0])
print("Arguments:", sys.argv[1:])
```

```
$ python my_script.py arg1 arg2
Script: my_script.py
Arguments: ["arg1", "arg2"] # Beware, these are
    strings !
```

There is the library `argparse` which handles more complex arguments (see web & *Going further* section in last *Practical session*)

Some exercises to finish with.

Exercise 10

Write a script `greet.py` that takes a name as argument and prints `Hello, <name>!`. Then modify it to also accept an optional second argument for a custom greeting (*e.g.* `python greet.py Alice Bonjour` prints `Bonjour, Alice!`). Handle the case where no argument is provided.

Exercise 11

Write a script `stats.py` that takes a list of numbers as arguments and prints their sum, min and max. For instance:

```
$ python stats.py 3 1 4 1 5 9
Sum: 23  |  Min: 1  |  Max: 9
```

Hint: remember that `sys.argv` is a list of strings...